

RIESGOS Y PLANES

ID	Descripción del Riesgo	Prob	Impacto	Prioridad
R01	Retraso en los Scripts SQL, tarea que bloquea toda la implementación de la Fase 3	Media	Alto	1
R02	Incompatibilidad entre Ian (Frontend) y Miguel (Backend) durante la integración	Alta	Alto	1
R03	Baja disponibilidad de un integrante en fechas críticas del camino crítico	Media	Alto	2
R04	Desconocimiento de tecnologías clave del stack	Media	Alto	2
R05	Pérdida de código por fallo o uso incorrecto del repositorio GitHub	Baja	Alto	3
R06	Problemas de configuración del entorno Docker entre los integrantes del equipo	Media	Medio	3

R01 — Retraso en los Scripts SQL

Este riesgo es el más crítico del proyecto. Los Scripts SQL (tarea 2.2.1, asignada a Rubén) son el primer paso de la Fase 3 y su retraso bloquea directamente el trabajo de Ian y Miguel, que no pueden implementar frontend ni backend sin la base de datos operativa.

Plan de prevención: Rubén debe tener el Diagrama ER completamente validado antes del fin de la Fase 2 (8 de mayo), de modo que los scripts puedan redactarse desde el primer día hábil de la Fase 3 (9 de mayo) sin necesidad de revisar el diseño. Además, se recomienda usar DB Fiddle durante la Fase 2 para validar sintaxis SQL y restricciones antes de la implementación real.

Plan de contingencia: Si los scripts sufren un retraso superior a un día, Ian y Miguel arrancan con datos mock para no perder tiempo de desarrollo. Rubén entrega los scripts por partes priorizando las tablas de USUARIO y NOTICIA, que son las que desbloquean los módulos de autenticación y noticias, los más críticos.

R02 — Incompatibilidad Frontend / Backend

Ian desarrolla las vistas y Miguel desarrolla la API. Si no acuerdan previamente el contrato de la API (endpoints, formato JSON, códigos de respuesta), la integración puede generar errores difíciles de depurar en los últimos días.

Plan de prevención: Antes de comenzar la implementación, Ian y Miguel definen y documentan un contrato de API básico (listado de endpoints, parámetros y respuestas esperadas) que queda registrado en el repositorio. Los Diagramas de Secuencia de la Fase 2 sirven como base para este contrato.

Plan de contingencia: Si durante la integración aparecen incompatibilidades, se realiza una sesión de trabajo conjunto (presencial o por videollamada) para alinear ambas partes. Se prioriza arreglar los módulos del camino crítico (Auth y Noticias) antes que los secundarios (Catálogo, Multimedia).

R03 — Baja disponibilidad de un integrante en fechas críticas

El equipo compagina este proyecto con otras asignaturas. Una baja por enfermedad, examen u otra causa en un integrante del camino crítico (especialmente durante la semana del 12 al 19 de mayo) puede comprometer la entrega.

Plan de prevención: Todas las tareas están documentadas en el WBS con suficiente detalle para que otro integrante pueda continuarlas. Cada desarrollador comenta su código desde el primer día y hace push al repositorio al final de cada sesión de trabajo, sin acumular cambios locales.

Plan de contingencia: Si un integrante cae en la semana crítica de implementación, el resto del equipo redistribuye sus tareas priorizando las del camino crítico. Daniel, cuyo trabajo en Fase 3 (Post-Mortem) tiene mayor margen temporal, puede apoyar con tareas de documentación o testing para liberar a Rubén.

R04 — Desconocimiento de tecnologías clave

El stack elegido incluye tecnologías que algún integrante puede no dominar completamente: Docker para el servidor, Python con Flask o similar para el backend, y los Service Workers para la PWA.

Plan de prevención: Durante la Fase 1 y el inicio de la Fase 2, cada integrante dedica tiempo a familiarizarse con la tecnología de su área mediante tutoriales y prototipos mínimos. Ian prueba la configuración básica de la PWA, Miguel levanta un contenedor Docker con una ruta de prueba, y Rubén valida las restricciones SQL en DB Fiddle.

Plan de contingencia: Si en la Fase 3 un integrante se bloquea con una tecnología, el equipo valora simplificar esa parte del sistema. Por ejemplo, si la PWA da problemas, se entrega la web responsive sin instalación en pantalla de inicio, documentando la limitación en el Post-Mortem.

R05 — Pérdida de código por fallo en el repositorio

Un uso incorrecto de Git puede provocar pérdida de trabajo.

Plan de prevención: La rama `main` está protegida y solo recibe código mediante Pull Requests desde `develop`. Ningún integrante hace push directo a `main`. Rubén exporta periódicamente el script SQL actualizado a Google Drive como copia de seguridad adicional.

Plan de contingencia: GitHub mantiene historial completo de commits, por lo que cualquier pérdida accidental se puede revertir recuperando commits anteriores. El equipo designa a Miguel como responsable de resolver conflictos críticos de Git dado su rol de integrador SCM.

R06 — Problemas de configuración del entorno Docker

Docker puede comportarse de forma diferente en Windows y macOS, generando el clásico problema de "en mi máquina funciona". Esto puede consumir horas de depuración que no están presupuestadas.

Plan de prevención: Miguel define y comparte el `docker-compose.yml` en el repositorio desde el inicio de la Fase 3. Todos los integrantes levantan el entorno en local antes del día 12 de mayo para detectar problemas con margen.

Plan de contingencia: Si Docker da problemas irresolubles en alguna máquina, ese integrante trabaja con el entorno del repositorio de Miguel como referencia, o se sustituye Docker temporalmente por XAMPP para no perder tiempo de desarrollo. La solución Docker se retoma cuando el bloqueo principal está resuelto.